



Laboratory Works

Web-Oriented Distributed System

1. спочатку **backend** як система
2. потім **асинхронність і події**
3. потім **AI / background**
4. лише після цього — **frontend**
5. orchestration & delivery — **оглядово**



BACKEND LABS



Backend Lab 1

Core Backend & Job Lifecycle



Мета

Навчитися будувати **backend як source of truth**, а не CRUD-API для UI.

Завдання

- Створити Node.js + Express backend
- Реалізувати **Job / Todo модель**

Реалізувати **job lifecycle**:

CREATED → QUEUED → PROCESSING → DONE / ERROR

-
- Зберігати дані в SQLite

Обмеження

- **×** без RabbitMQ
- **×** без AI
- **×** без WebSocket

Результат

- REST API
 - Jobs зберігаються
 - Стани змінюються коректно
-

Backend Lab 2

Authentication & Authorization (JWT)

Мета

Навчитися ізолювати користувачів, не ускладнюючи систему.

Завдання

- Реалізувати видачу JWT
- JWT використовується для ідентифікації
- Middleware:
 - перевірка токена
 - витяг `userId`
- Jobs одного користувача недоступні іншому

Обмеження

-  без реєстрації
-  без паролів

Результат

- Backend працює з `userId`
 - Дані ізольовані
-

Backend Lab 3

Message Broker & Async Processing

Мета

Навчитися **відокремлювати довгі операції** від HTTP.

Завдання

- Підключити RabbitMQ
- Backend:
 - публікує `transcription.request`
 - оновлює статус job
- Реалізувати **ідемпотентність** job

Обмеження

- ❌ без AI
- ❌ без S3

Результат

- Jobs переходять у `QUEUED`
- Backend не блокується

Прийнято 👍

Тоді **переформулюємо все як ОДНУ, цілісну 4-ту лабораторну**, без поділу на 4–5, але з **внутрішньою еволюцією складності** (stub → real heavy-processing).

Нижче — **фінальна версія Lab 4**, готова для силлабуса.

Backend Lab 4

Background & Async Processing (Non-HTTP Services)

Мета

Навчитися будувати **складні веб-орієнтовані backend-рішення**, у яких:

- довгі операції виконуються **асинхронно**
 - heavy-processing винесений в **окремий background сервіс**
 - backend **не блокується** і не залежить від тривалості задач
 - взаємодія між сервісами відбувається **без HTTP**, через події
-

Ключові ідеї лабораторної

- non-HTTP сервіси
 - event-driven architecture
 - background workers
 - асинхронні workflow
 - ізоляція heavy-processing
 - стабільність backend-сервісів
-

Архітектурна схема

Backend (Node.js)

|

| publish job

v

RabbitMQ

|

v

Background Service (Python)

|

| publish events (progress / completed / failed)

v

RabbitMQ

|

v

Backend (update job status)

-  HTTP використовується **лише для керування**,
 -  виконання задачі — **поза HTTP**.
-

Завдання

Background Service (Python)

- підключатися до RabbitMQ
- **consume повідомлення з черги**
- виконувати довгу операцію:
 - на першому етапі — **sleep** або stub
 - на фінальному етапі — **реальний heavy-processing**
- публікувати події:

- progress
- completed
- failed

Backend (Node.js)

- створювати job
- публікувати job у RabbitMQ
- **реагувати на події** з RabbitMQ
- оновлювати статус job у БД
- залишатися стабільним незалежно від тривалості обробки

Обмеження (критично важливо)

Background Service (Python)

-  НЕ має REST API
-  НЕ приймає HTTP-запитів
-  НЕ знає про frontend
-  комунікація **виключно** через RabbitMQ

Backend

-  не виконує heavy-логіку
-  не чекає результат синхронно
-  лише координує процес

Job lifecycle (мінімум)

CREATED
PROCESSING
DONE
ERROR

 Зміна статусів відбувається **тільки** через події.

Heavy-processing (фінальна частина лаби)

Студент **сам** обирає тип обробки, наприклад:

- ML / AI модель
- обробка зображень
- обробка відео або аудіо
- генерація великих даних
- складні обчислення

 Обробка має:

- тривати відчутний час
- бути винесеною в Python-сервіс
- не блокувати backend

Очікуваний результат

- асинхронний workflow працює
- job коректно проходить весь lifecycle
- backend отримує:
 - прогрес
 - фінальний результат
- система стабільна навіть при довгих задачах

Backend Lab 5

Object Storage (S3 / MinIO)

Мета

Зрозуміти, що:

База даних ≠ сховище файлів

Завдання

- Підключити S3-compatible storage (MinIO)
- AI сервіс:
 - зберігає результат у S3
- Backend:
 - зберігає лише **s3Key**
 - віддає результат по запиту

Результат

- SQLite → metadata
 - S3 → результат
-

Backend Lab 6

WebSocket Backend

Мета

Навчитися **push-моделі** в web-системах.

Завдання

- WebSocket сервер
- Авторизація через JWT
- Push:
 - progress
 - status update
 - completion

Обмеження

-  frontend ще не обов'язковий
(можна тестувати через ws-client)

Результат

- Backend надсилає події в реальному часі
-

FRONTEND LABS (як клієнт системи)

Frontend Lab 7

React as a Client

Мета

Показати, що frontend — **не центр архітектури**.

Завдання

- React UI:
 - створення job
 - список job
 - статуси
- REST API

Результат

- UI відновлює стан після refresh
 - Backend лишається source of truth
-

Frontend Lab 8

Real-Time Frontend (WebSocket)

Мета

Навчитися працювати з **реальним часом**.

Завдання

- WebSocket client
- JWT авторизація
- Reconnect logic
- Fallback на REST

Результат

- UI оновлюється без перезавантаження
 - Користувач бачить прогрес
-



ПОЗА ЛАБАМИ (БОНУСИ)

Orchestration & Containerization

- Docker
- docker-compose
- service separation

Delivery & CI/CD

- GitHub Actions
- AWS EC2
- env-based configuration